



Histoire du Logiciel Libre

Partie 1 : 1950-1990

Session 2 – Cours M1 Open Source Software



Objectifs de la séance



Ce que vous saurez faire

À la fin de cette séance, vous serez capables de :

- **Retracer** l'évolution de l'industrie du logiciel des années 50 aux années 90
- **Expliquer** le contexte d'émergence du mouvement du logiciel libre
- **Comprendre** la philosophie Unix et son influence
- **Identifier** les figures fondatrices et leurs contributions
- **Analyser** les arguments du GNU Manifesto



Partie 1

La préhistoire du logiciel (1950-1970)

L'ère des mainframes

Le contexte

- Ordinateurs = machines géantes et coûteuses
- Acheteurs = gouvernements, universités, grandes entreprises
- Le logiciel est un **accessoire** du matériel
- Programmation sur mesure par le client

Les acteurs

- IBM domine le marché
- DEC (miniordinateurs)
- Universités : MIT, Berkeley, Bell Labs
- Les "hackers" du MIT (sens original)

Le logiciel : un bien partagé

Dans les années 60, le partage du code est **la norme** :

- Le logiciel est **inclus** avec le matériel
- Les utilisateurs **échangent** leurs programmes
- IBM distribue le code source
- Les groupes d'utilisateurs (SHARE, DECUS) partagent des améliorations

"Le logiciel était comme une recette de cuisine : on la partageait naturellement."

1969 : Le tournant IBM

En 1969, IBM annonce le **unbundling** :

- Séparation matériel / logiciel
- Le logiciel devient un **produit commercial** distinct
- Début de l'industrie du logiciel propriétaire

Pourquoi ?

- Pression antitrust du gouvernement américain
- Opportunité commerciale identifiée

Le logiciel passe du statut de "bien commun" à "propriété commerciale".



Partie 2

Unix : naissance d'un écosystème



La chronologie d'Unix

Unix : les origines (1969)

Ken Thompson et **Dennis Ritchie** créent Unix aux Bell Labs (AT&T)

Contexte

- 1965 : MIT, Bell Labs, GE lancent MULTICS
- 1969 : Bell Labs se retire du projet
- Thompson et Ritchie créent UNICS sur un PDP-7
- 1972 : Ritchie conçoit le langage C
- 1973 : Unix réécrit en C (portabilité)

Pourquoi AT&T distribue librement ?

- Consent Decree de 1956 : AT&T ne peut commercialiser que la téléphonie
- Unix distribué aux universités avec le code source
- Coût : une bande magnétique + frais d'envoi

Caractéristiques d'Unix

Innovations techniques

- Système simple et élégant
- Portable (écrit en C)
- Multi-utilisateur, multi-tâches
- "Everything is a file"
- Processus et mémoire virtuelle

Diffusion

- AT&T ne peut pas commercialiser (monopole télécom)
- Distribué aux universités avec le code source
- Devient le standard académique
- Fork BSD à Berkeley (1977)



Architecture Unix

Kernel : Contrôle le système. Gestion des processus, communication avec les périphériques, système de fichiers.

Shell : Interface utilisateur. Ensemble d'utilitaires, création de scripts.

Outils : Le plus grand ensemble. Édition, traitement de texte, développement, communication...



Partie 3

La philosophie Unix



L'invention des pipes (1973)

"Was the notion of toolbox there before pipes?" – Mahoney

"No." – McIlroy

"Or did pipes create it?"

"Pipes created it."

L'invention des pipes par Doug McIlroy a donné naissance à la philosophie Unix : **des programmes qui font une seule chose bien et qui travaillent ensemble.**



Doug McIlroy (1978)

Les règles fondamentales de la philosophie Unix :

1. **Make each program do one thing well.** To do a new job, build afresh rather than complicate old programs.
2. **Expect the output of every program to become the input to another.** Don't clutter output with extraneous information.
3. **Design and build software to be tried early**, ideally within weeks. Don't hesitate to throw away clumsy parts.
4. **Use tools** in preference to unskilled help to lighten a programming task.

Les règles d'Eric S. Raymond

- **Modularity:** Write simple parts connected by clean interfaces
- **Clarity:** Clarity is better than cleverness
- **Composition:** Design programs to be connected to other programs
- **Simplicity:** Design for simplicity; add complexity only where you must
- **Transparency:** Design for visibility to make inspection easier
- **Robustness:** Robustness is the child of transparency and simplicity
- **Least Surprise:** In interface design, always do the least surprising thing
- **Silence:** When a program has nothing surprising to say, it should say nothing

"Worse is Better"

Concept formulé par Richard Gabriel (1989) :

The MIT/Stanford approach

- L'interface doit être correcte
- L'implémentation peut être complexe
- La cohérence est primordiale

The New Jersey approach (Unix)

- La simplicité de l'implémentation prime
- L'interface peut être imparfaite
- "Good enough" gagne

■ "Unix and C are the ultimate computer viruses." – Richard Gabriel

Cette philosophie explique pourquoi Unix a conquis le monde malgré ses imperfections.

Berkeley et BSD

La connexion Berkeley

- 1976-77 : Ken Thompson en sabbatique à UC Berkeley
- Bill Joy et Chuck Haley démarrent BSD
- 1979 : CSRG (Computer Systems Research Group)
- Contrat DARPA pour le développement

Contributions majeures de BSD

- Pile TCP/IP (BSD 4.2, 1983)
- vi, csh, sendmail
- Virtual memory
- Sockets pour le réseau
- Base de SunOS, macOS ...

La culture hacker du MIT

Au MIT, une communauté de programmeurs développe une culture unique :

Les valeurs

- Partage du code
- Méritocratie technique
- "L'information veut être libre"
- Amélioration collaborative

Le laboratoire d'IA

- Système ITS (Incompatible Timesharing System)
- Pas de mots de passe
- Tout le code est partagé
- Richard Stallman y travaille dès 1971



Partie 4

La fermeture du logiciel (1970-1983)



L'essor du logiciel commercial

Année	Événement
-------	-----------

1975	Microsoft est fondé (Bill Gates, Paul Allen)
1976	"Open Letter to Hobbyists" de Bill Gates
1976	Apple est fondé
1979	VisiCalc : premier "killer app"
1980	Copyright Act inclut explicitement le logiciel
1981	IBM PC avec MS-DOS
1982	AT&T peut commercialiser Unix (après éclatement)

La lettre ouverte de Bill Gates (1976)

"As the majority of hobbyists must be aware, most of you steal your software. [...] Who can afford to do professional work for nothing? What hobbyist can put 3-man years into programming, finding all bugs, documenting his product and distribute for free?"

Arguments de Gates :

- Le développement logiciel est coûteux
- Le "piratage" empêche l'innovation
- Les développeurs méritent d'être payés

Cette lettre marque un tournant : le logiciel devient explicitement une marchandise.

L'effondrement du Lab d'IA du MIT

Au début des années 80, le laboratoire d'IA du MIT se vide :

Ce qui se passe

- Spin-offs commerciaux (Symbolics, LMI)
- Le code devient propriétaire
- Les hackers partent
- La communauté se disloque

L'incident de l'imprimante Xerox

- Stallman veut modifier le driver
- Xerox refuse de donner le code source
- "Je me suis senti trahi"
- Déclat pour Stallman

L'imprimante Xerox 9700

Cet incident est le **mythe fondateur** du mouvement du logiciel libre.

Stallman voulait simplement ajouter une notification quand le papier était coincé. Mais sans le code source, c'était impossible.

"J'ai réalisé que le logiciel propriétaire était une injustice sociale."



Partie 5

Richard Stallman et la naissance du libre



Richard Stallman

Parcours

- Né en 1953 à New York
- Programmeur au MIT AI Lab depuis 1971
- "Dernier survivant" de la culture hacker
- Brillant développeur (Emacs)

Vision

- Le logiciel propriétaire est une injustice
- Les utilisateurs méritent la liberté
- Il faut recréer un écosystème libre
- C'est une question d'**éthique**, pas de pragmatisme



Vidéo : RMS explique les libertés

[Cliquez pour voir la vidéo : RMS "Liberté, Égalité, Fraternité"](#)

(Ouvrir avec VLC)

■ "Liberté, Égalité, Fraternité" – Richard Stallman



27 septembre 1983 : l'annonce

Stallman poste sur Usenet :

"Starting this Thanksgiving I am going to write a complete Unix-compatible software system called GNU (for Gnu's Not Unix), and give it away free to everyone who can use it."

Objectif : Créer un système d'exploitation entièrement libre

Stratégie : Compatible Unix pour faciliter l'adoption

Le projet GNU

Composants développés

- **GNU Emacs** (1984) : éditeur de texte
- **GCC** (1987) : compilateur C
- **GDB** : débogueur
- **GNU Make** : outil de build
- **Bash** : shell
- Utilitaires (ls, cat, grep...)

Ce qui manquait

- Le noyau (kernel)
- Projet GNU Hurd lancé en 1990
- Retardé par des choix techniques ambitieux
- Ne sera jamais vraiment terminé

Le GNU Manifesto (1985)

Document fondateur publié dans Dr. Dobb's Journal.

Arguments principaux :

1. **Les utilisateurs seront libres** de partager et modifier
2. **Le logiciel sera meilleur** grâce à la collaboration
3. **Les développeurs pourront toujours gagner leur vie** (services, support)
4. **C'est une question d'éthique** : le logiciel propriétaire divise les utilisateurs

"I consider that the Golden Rule requires that if I like a program I must share it with other people who like it."

La Free Software Foundation (1985)

Stallman fonde la FSF pour :

Missions

- Promouvoir le logiciel libre
- Financer le développement de GNU
- Employer des développeurs
- Défendre juridiquement les licences

Réalisations

- Définition des 4 libertés
- Création de la licence GPL
- Lobbying politique
- Certifications de licences

La GPL : le "hack juridique"

1989 : Publication de la GNU General Public License (GPL)

Le mécanisme du copyleft :

1. Le code est protégé par le droit d'auteur (copyright)
2. La licence **utilise** ce droit pour **garantir** les libertés
3. Toute version modifiée doit être redistribuée sous GPL
4. Les libertés sont **virales** et irrévocables

"Copyleft" : utiliser le copyright contre lui-même pour garantir la liberté.

1985 : X Window System

Le **X Consortium** au MIT crée et distribue le système de fenêtrage X11.

```
From: rws@mit-bold (Robert W. Scheifler)
Subject: window system X
Date: 19 Jun 1984
```

```
I've spent the last couple weeks writing a window system for the VS100.
I stole a fair amount of code from W, surrounded it with an asynchronous
rather than a synchronous interface, and called it X.
```

```
[ ... ]
```

```
Anyone who wants the code can come by with a tape.
```

X Window utilise une licence **très permissive** (MIT License) – un autre modèle de logiciel libre.

Fin des années 80 : l'écosystème s'étoffe

Année Événement

1987	GCC 1.0 – compilateur libre de qualité
1987	Perl 1.0 (Larry Wall)
1988	X Window System devient libre (MIT)
1989	GPL version 1
1989	Cygnus Solutions : première entreprise GNU
1990	Début du projet GNU Hurd

En 1990 : Tous les composants d'un système libre existent... sauf le noyau.



Résumé



Ce qu'il faut retenir

1. **Avant 1969** : Le logiciel était naturellement partagé
2. **Unix (1969)** : Crée une philosophie de développement influente
3. **1970-1983** : Fermeture progressive, émergence du logiciel propriétaire
4. **1983** : Richard Stallman lance le projet GNU
5. **1985** : Création de la FSF et publication du GNU Manifesto
6. **1989** : La GPL crée un mécanisme juridique pour protéger les libertés
7. **1990** : L'écosystème GNU est presque complet, il manque le noyau

Timeline récapitulative

Année	Événement clé
1969	Création d'Unix, unbundling IBM
1973	Invention des pipes (McIlroy)
1976	Lettre ouverte de Bill Gates
1977	BSD à Berkeley
1980	Copyright Act américain
1983	Annonce du projet GNU
1985	FSF créée, GNU Manifesto
1987	GCC 1.0
1989	GPL v1
1991	??? (suite au prochain épisode)



Pour aller plus loin

Lectures :

- Peter Salus, *"A Quarter Century of Unix"* (1994)
- Eric S. Raymond, *"The Art of Unix Programming"* (2003)
- Le GNU Manifesto : <https://www.gnu.org/gnu/manifesto.html>

Vidéos :

- Rob Pike, "Unix History" (2018) :
https://www.youtube.com/watch?v=_2NI6t2r_Hs
- Brian Kernighan on Unix : <https://lwn.net/Articles/881431/>



Pour la prochaine séance

Séance 3 : Histoire du logiciel libre (1990-aujourd'hui)

Préparation suggérée :

- Lire "The Cathedral and the Bazaar" (introduction) :
<http://www.catb.org/~esr/writings/cathedral-bazaar/>
- Qui est Linus Torvalds ?



Questions ?

Passons au TD